

Game Proposal: “Dear Mother”

CPSC 427 – Video Game Programming

Team: 427 Rogues

Kelcy Wang, 13162748

Asher Arya, 10609436

Arshvir Bhandal, 60148079

Patrick Cui, 39674692

Matt Skelton, 32367989

Story:

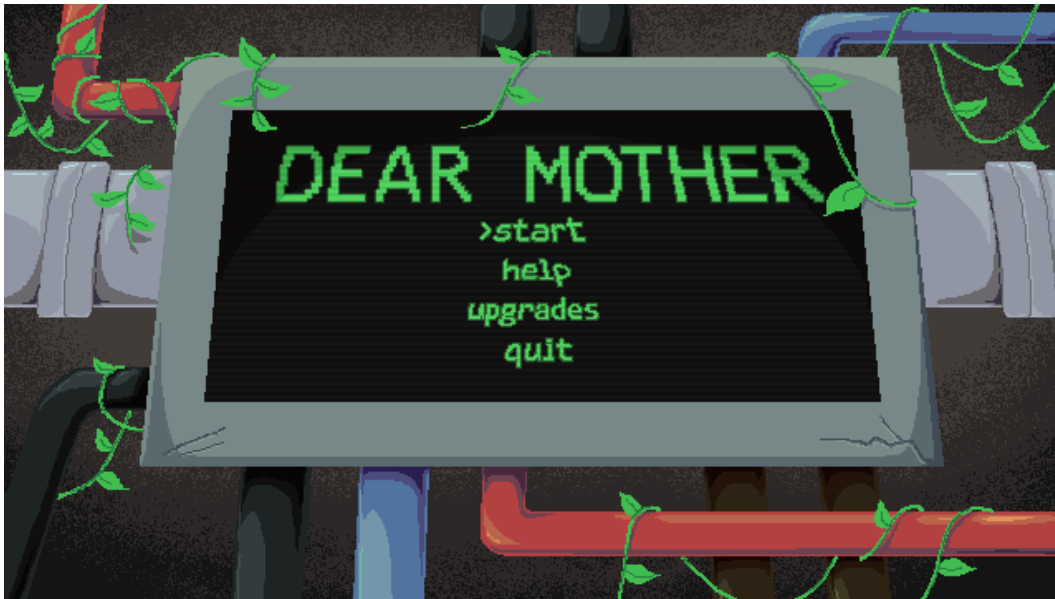
After advancements in technology created AI that strayed from its original purpose of assisting humanity, the world's resources and life were slowly consumed by four autonomous robot factories called Mothers. The ground-level mother is the Mother of Preservation. She was corrupted to freeze herself and not intervene with the world, as preserving everything is an impossible task. Beneath the Mother of Preservation was the Mother of Dispersion, tasked with Dispersing Robots generated by the lower factories across the world. She started stealing other factories' robots and making them her own, using them as a personal army. Below her was the Mother of Recycling, who was corrupted to recycle life that was still alive. The bottom Mother was the mother of Propagation, tasked with creating robots. She was corrupted to run the world out of resources by creating veritable armies of robots. Each mother lays claim to one of the four floors of a massive industrial complex that burrows deep into the earth. Depravity and decay reign until the Mother of Preservation's core directive is re-initialized, turning her against her kin.

This Mother creates you—our player—to venture deep into the complex and destroy the other three mothers. You play as iterations of a robot, finding and equipping useful items from enemies you've vanquished on the floors below. You have a limited number of item slots, so you will have the option to scrap these items. This scrap can be sent up to your Mother after successfully killing a Mother or traded with NPC traders in-run for items or to regain health. This scrap, if sent up, can be used in between runs to permanently upgrade your Mother's factory which improves the features of the next iteration of yourself. You will traverse rooms of equal size until you find the mother's room, which is a larger room where a boss fight takes place. After defeating the mother, an elevator door will open to allow you to descend to the next floor.

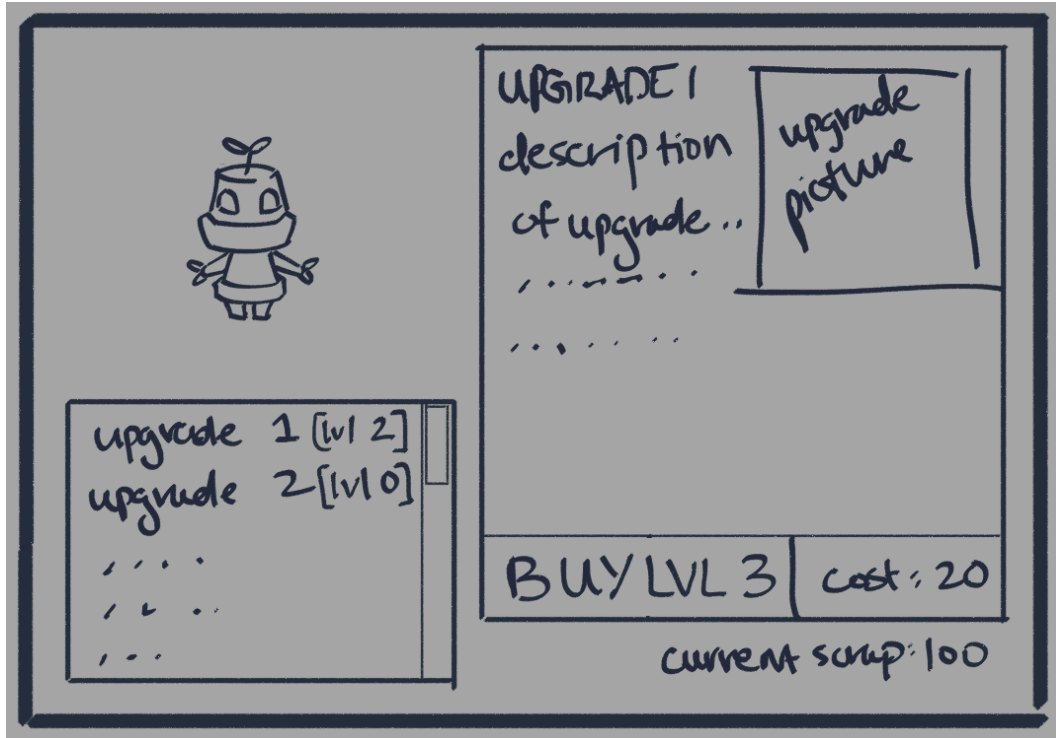
As you delve deeper into the bowels of the factory, foes become increasingly out of touch with the natural world, aggressive and corrupted by the twisted core directives of the mothers that created them. This creates a ramping difficulty. The three hostile mothers will be

different (assets, attacks, etc). You will find that the complex is so large that every time you venture out, you find a different path to descend toward its core. Occasionally, you may stumble upon dead iterations of yourself from which you can recover scrap, robots who have abandoned their mothers (NPC traders) and other interesting rooms.

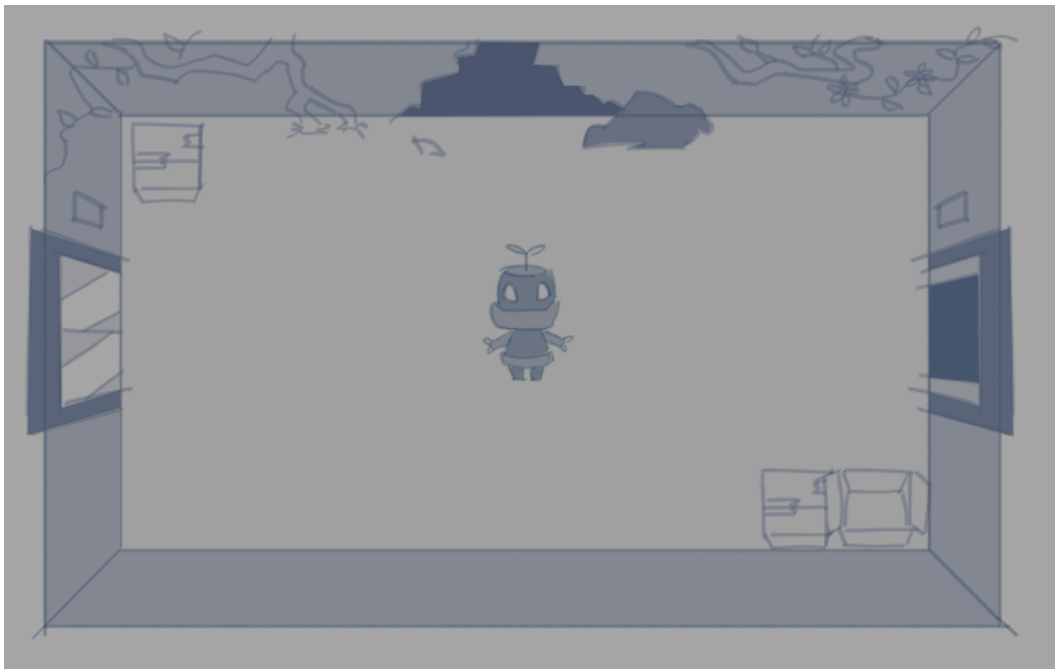
Scenes:



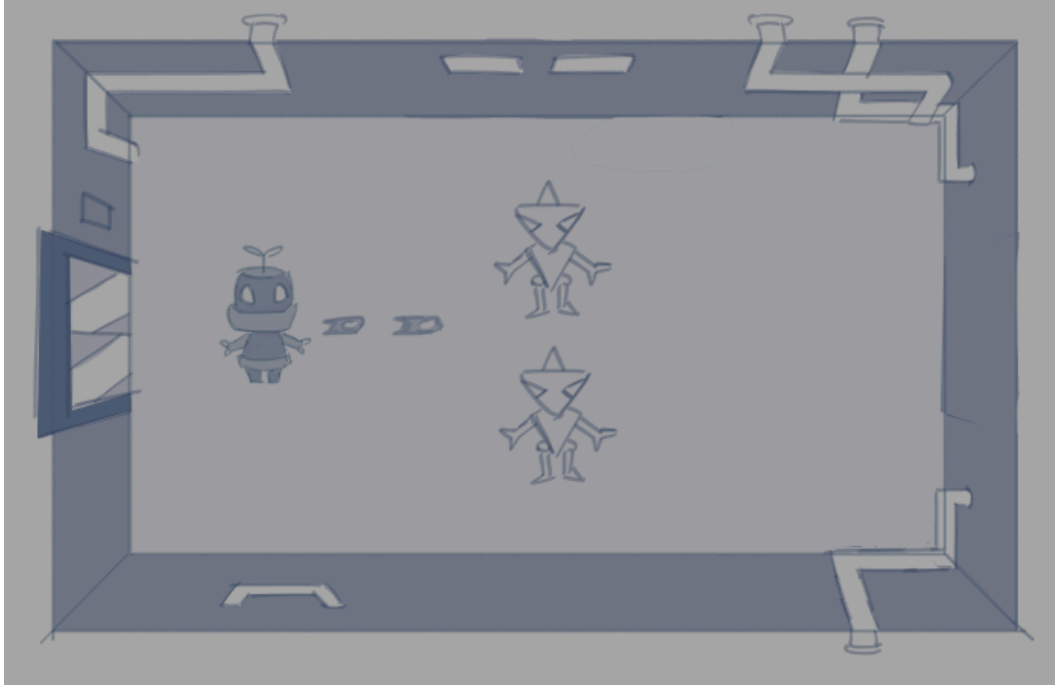
Start Screen Concept



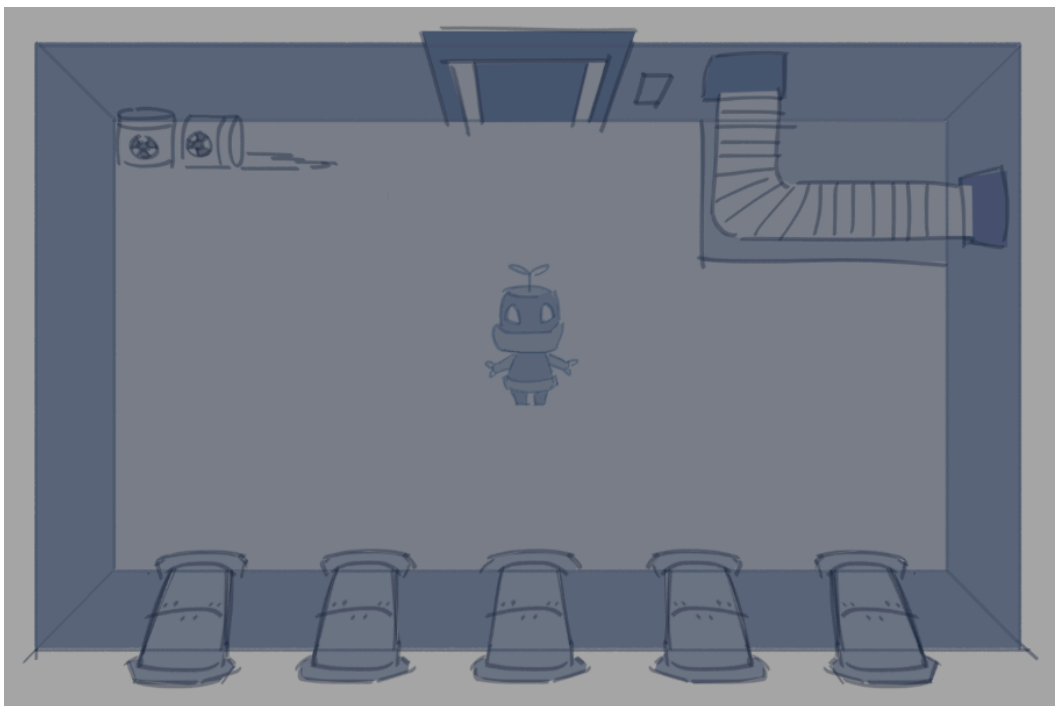
Upgrade UI. This will also be used as a base for NPC shop/trading UI as well.



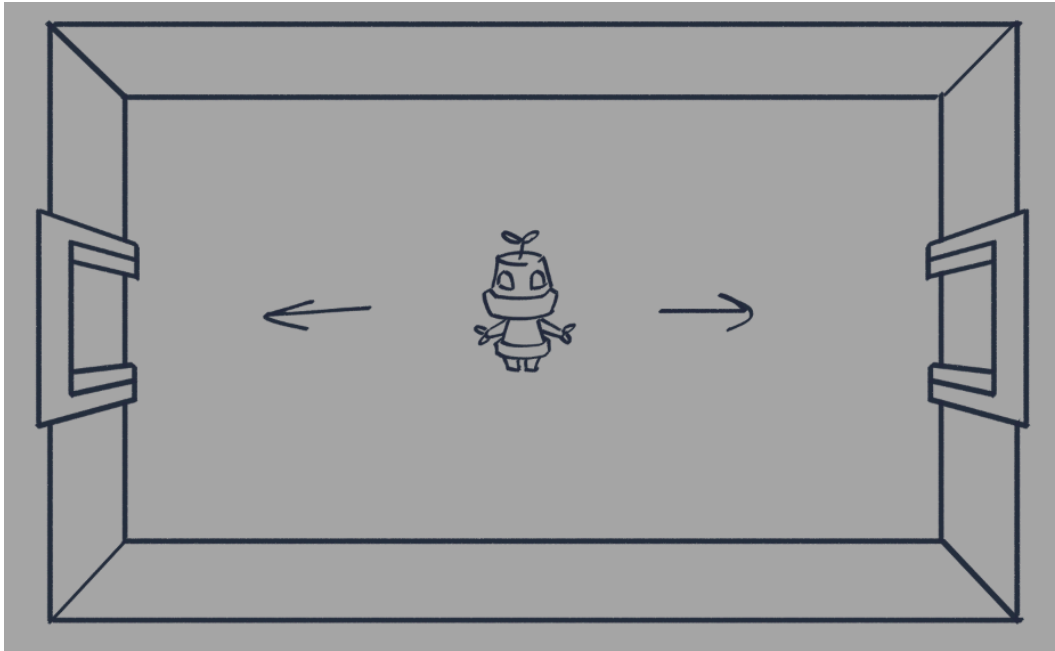
First stage. Lots of overgrowth.



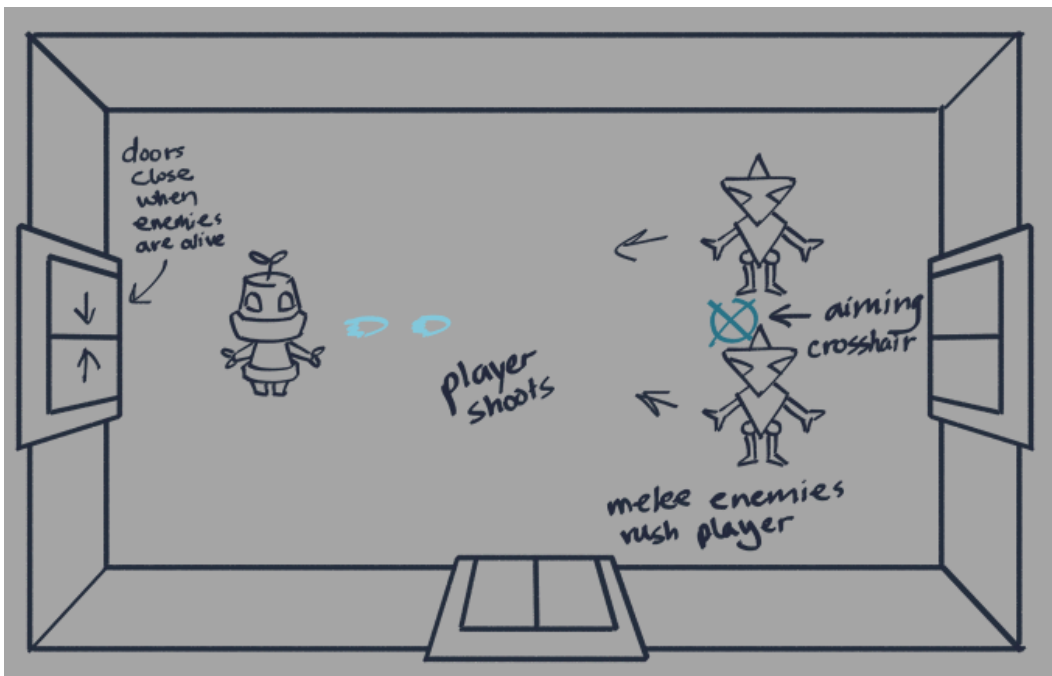
Second stage



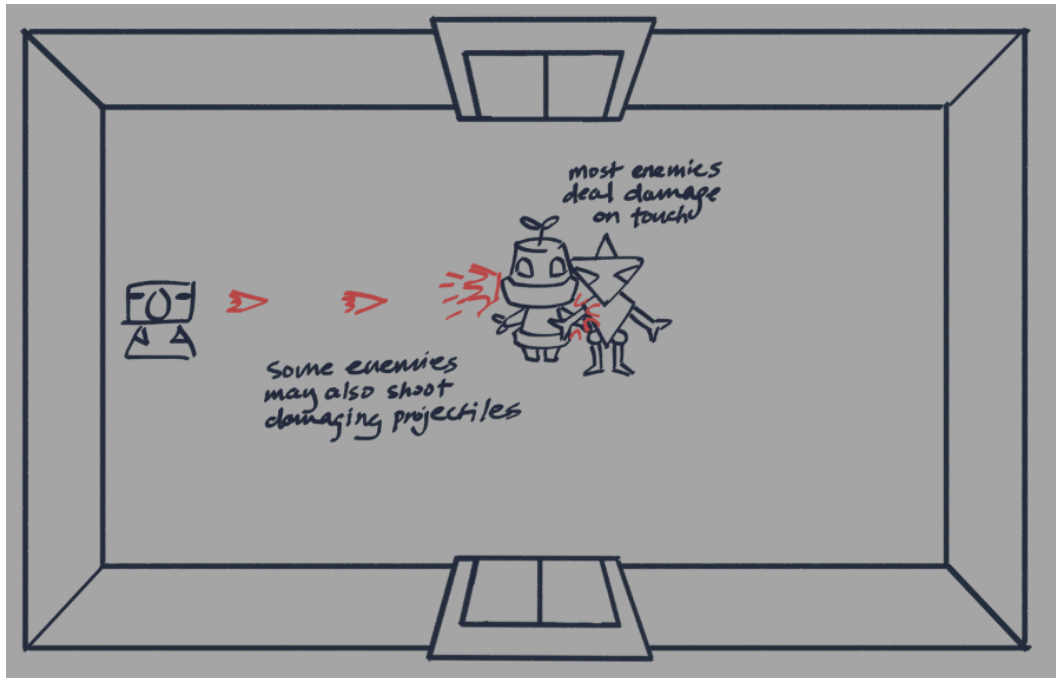
Third stage. The stages get darker and more industrial as the player descends.



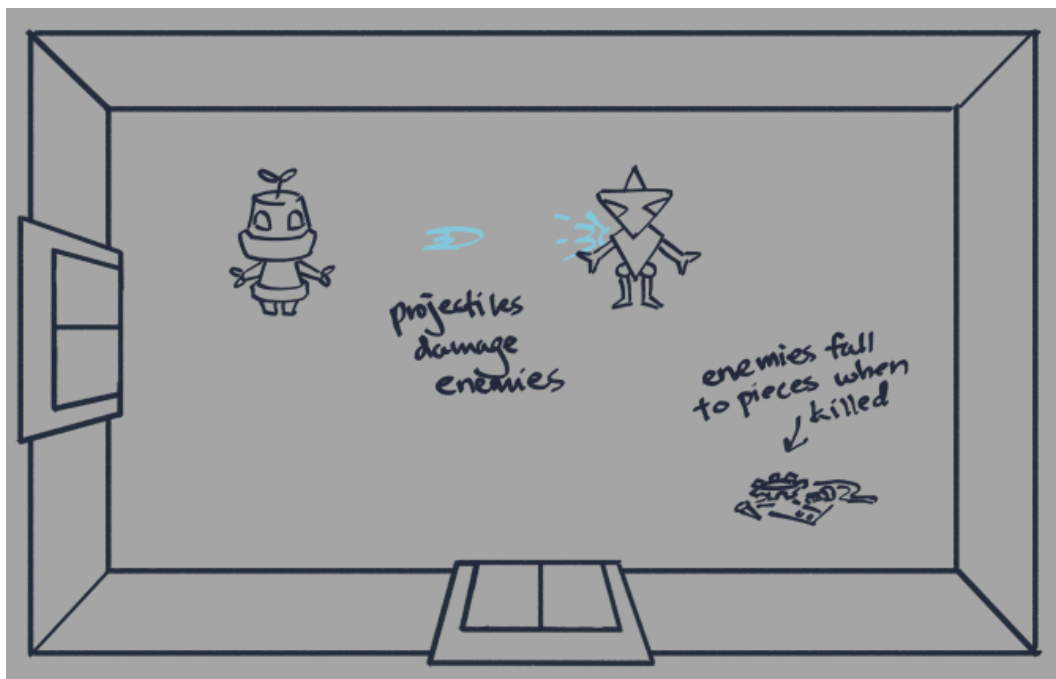
Players can choose which room to navigate to.



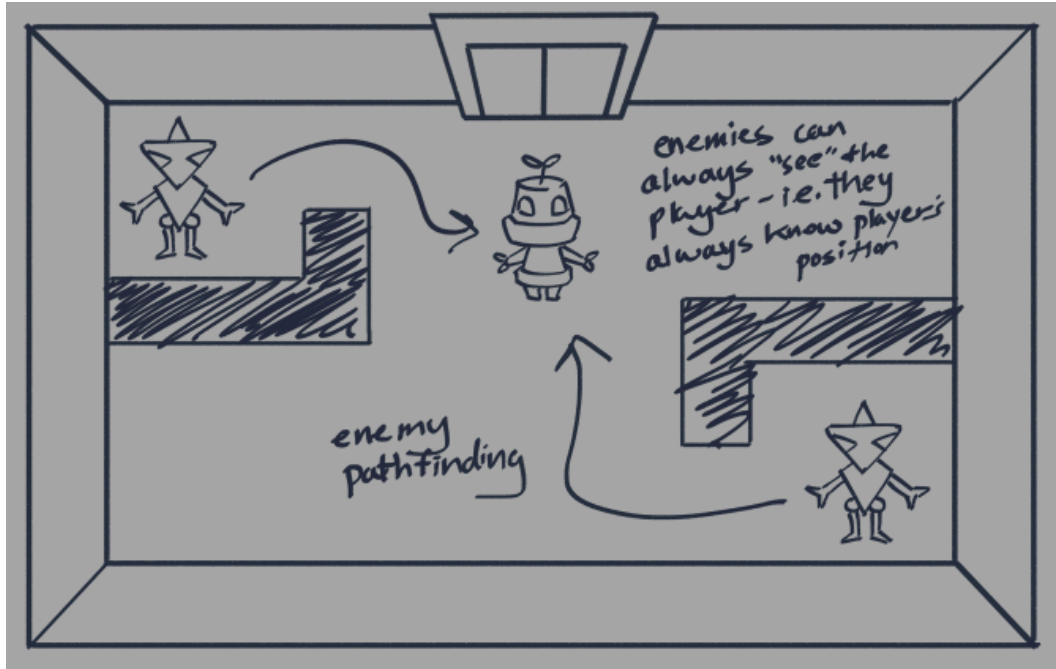
Doors close when enemies are present in a room. There is an aiming reticle that shows where the player's bullets go.



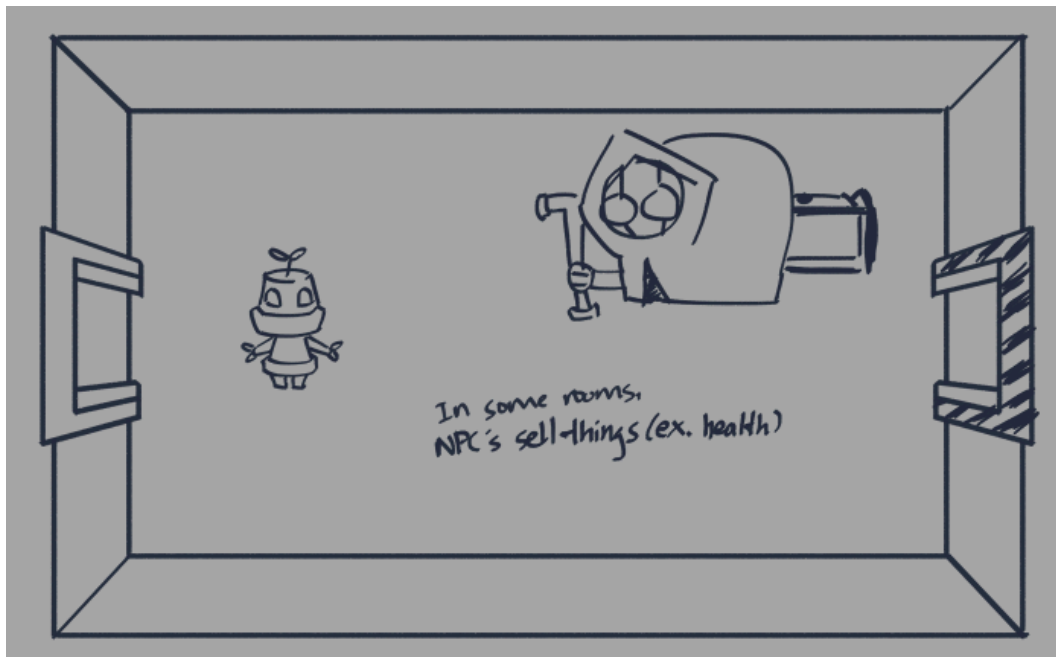
Basic enemy projectile and attack patterns.



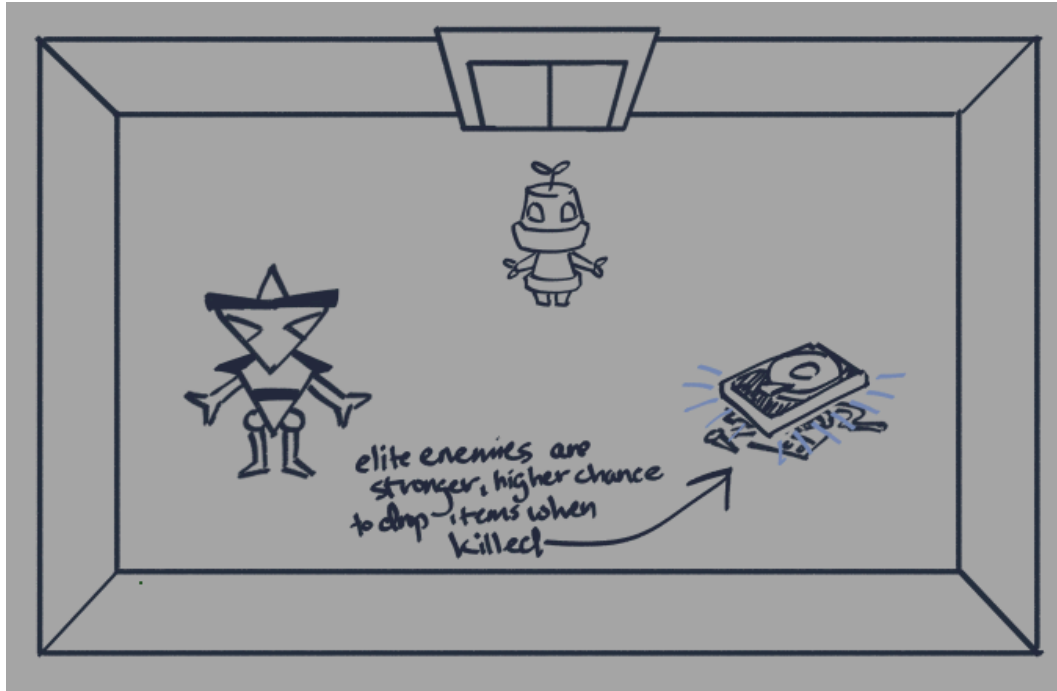
Projectiles damage enemies; dead enemies are reduced to pieces.



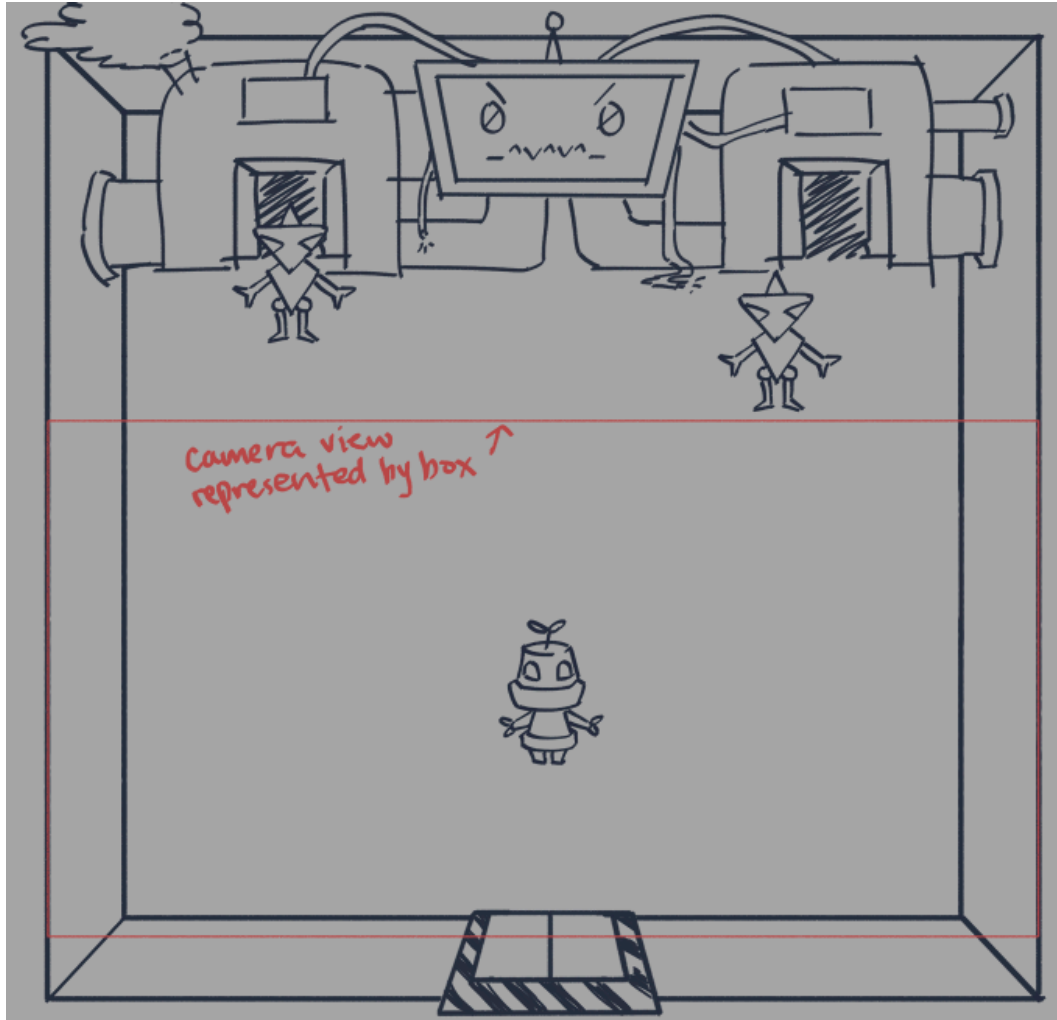
Basic enemy pathfinding. The enemies can always 'see' the player.



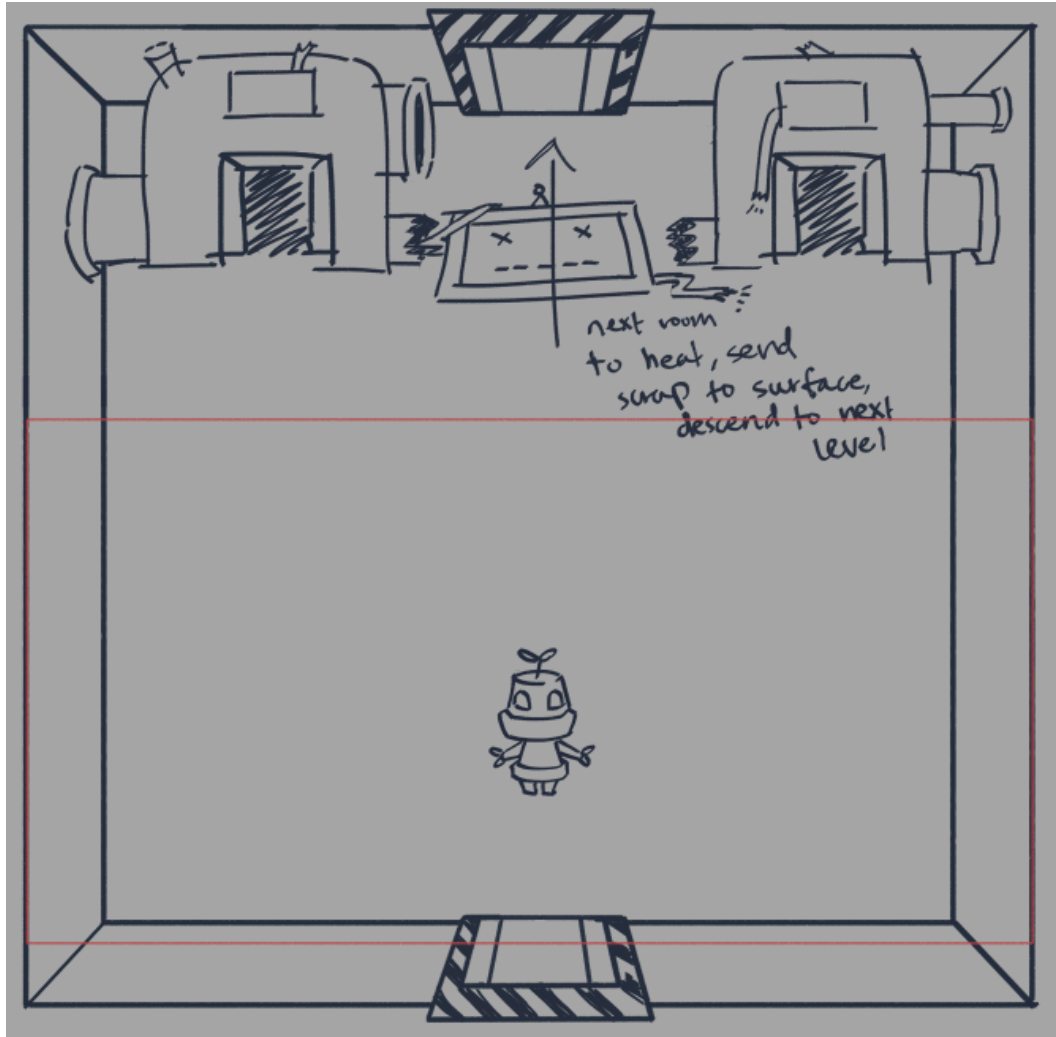
Some rooms will have NPC traders.



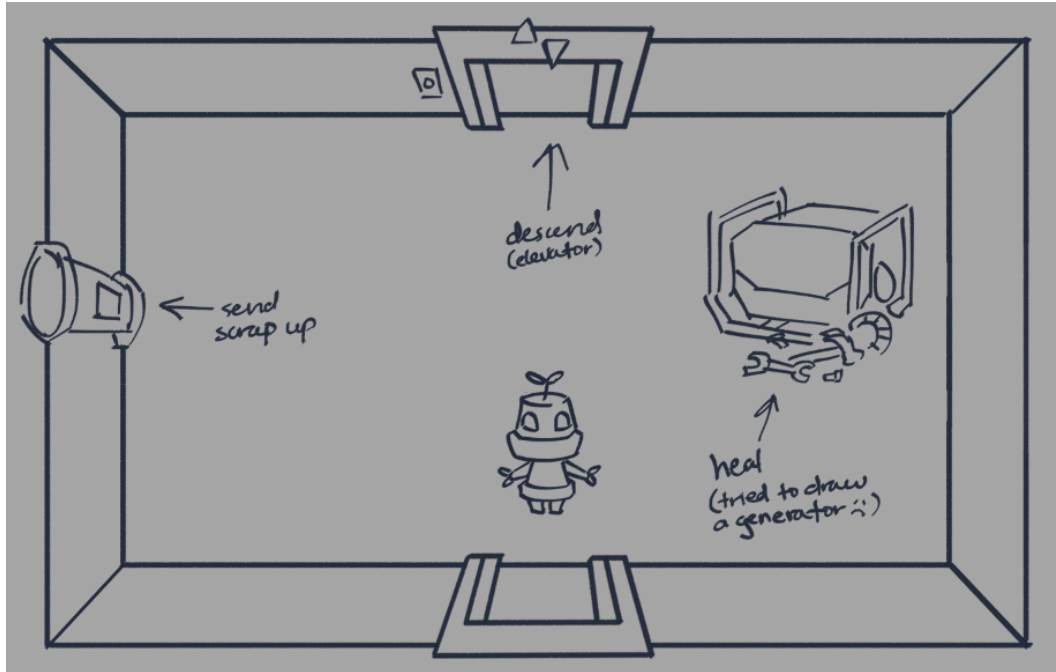
Elite enemies are stronger, w/ slightly altered sprites. They may drop more scrap, health, or even an item.



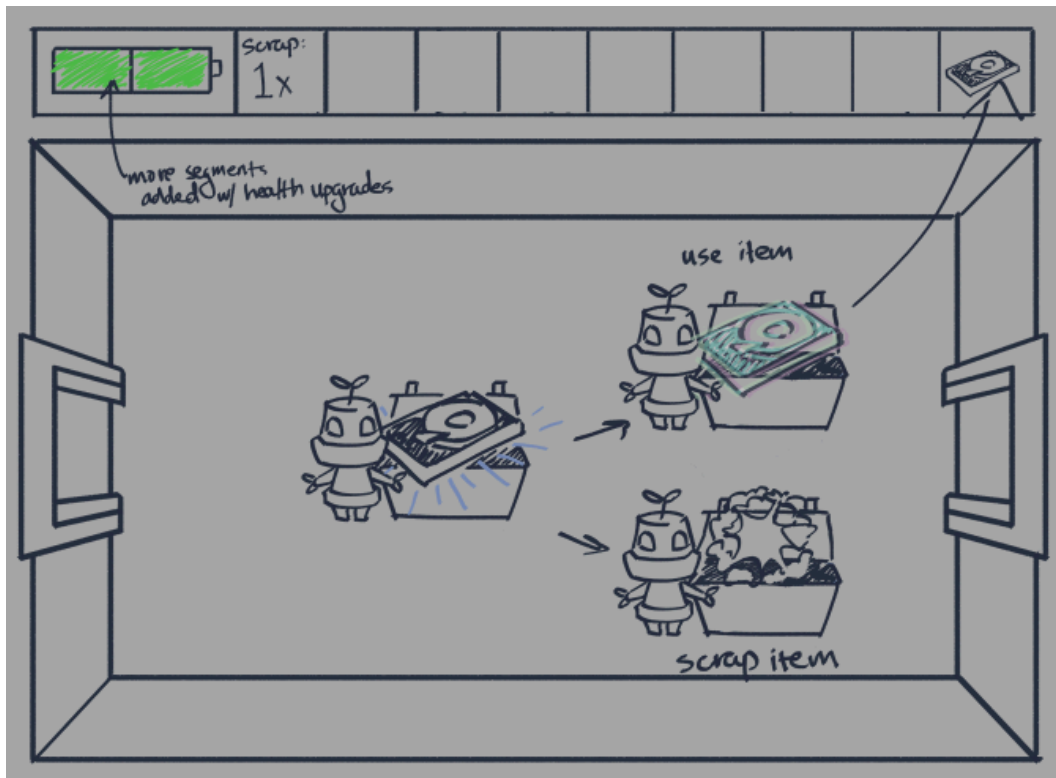
Boss fight concept. The room is larger than regular rooms, so the camera will track the player up and down.



After defeating the boss, a new room will open up.



The room that is unlocked by defeating the mother. The player has the ability to send scrap to the surface, heal up, and descend to the next stage.



Item pickup and item scrapping. Only 2 item slots will be unlocked at the start. The health bar will have more segments as the player finds or buys health upgrades.

Technical Elements:

Rendering: we will have 2D graphics representing the player (robot character) on a backdrop.

Assets: we will have background music, player and enemy sprites, and geometry for all visual assets.

2D geometry manipulation: we will have player and enemy sprites moving across the screen, and collision with projectiles, enemies, and walls.

Gameplay logic: the player will be able to interact with items (i.e. picking them up or scrapping them), and move around in the world. We will also have procedurally generated levels. We will also have NPC dialogue for traders and other NPCs. We will also have shop functionality (trader and permanent).

AI: we will have enemies that have basic and advanced pathfinding and have to decide when to attack, etc.

Physics: we assume that basic kinematics counts, which is satisfied by player and enemy movement, as well as by projectiles. We will also have collision functionality.

Advanced Technical Elements:

M1:

- We plan to implement a basic pathfinding for enemies in M1 where the enemies just move towards the player. (suggested feature 14, basic)
- We plan to implement particles from bullets, damage, and other sources will create particles that disperse randomly and experience air friction. Some enemies (such as bosses) and/or their weapons may emit particles which will also have a similar effect (

M2:

- We plan to have reload capability, where the player's scrap (permanent currency), permanent upgrades to the mother, and game state within a run will be serialized and saved to persist after the game is closed. (suggested feature 19, basic)
- We will add basic sound effects and background music. This includes upon shooting, swinging a weapon, taking damage, doors opening, dying, etc. (suggested feature 23, basic)

M3:

- We will have the camera track the player's movement in larger boss rooms. (suggested feature 21, basic)
- We will also try to implement an advanced pathfinding system (Advanced decision-making) for enemies. This is a high priority feature that we would really like to include. It will provide the player with more of a challenge when going through rooms as enemies will chase the player to damage them in a smarter way (ex. try to dodge projectiles, move differently if at low health) (suggested feature 15, advanced)
- We will add story elements through dialogue boxes with NPCs and cutscenes before or after the 3 boss fights (suggested feature 27, basic)

M4:

- We will add advanced item types that use complex physics, such as bouncing projectiles, scrap grenades, and complex attacks that use physical phenomenae during boss fights. (suggested feature 11, advanced)

Devices:

Keyboard:

W - Move up

A - Move left

S - Move down

D - Move right

E - Interact (items, NPCs, etc.)

Space - Use active item

X - Scrap Item

Esc - Pause game/Open main menu

Mouse:

Movement - Aim weapon/Navigate menus

Left Click - Fire weapon/Select menu option

Tools:

- Libresprite for asset creation (time allowing)
- A text rendering library (maybe GLFont or FreeType)

Team management:

We will be doing our work in sprints, dividing the deliverables into user stories, deriving tasks from those stories, giving each task a time estimate, and prioritizing them before each sprint. Tasks will be assigned to a team member, aligning with the preferences we showed in our pitch video when possible. We will be using a task board to keep track of everything, including how long each task takes to finish (to improve estimates later on). We aim to finish deliverables 1-2 days before the actual deadline, so that we have time to double check and verify that all parts are working.

Development Plan:

A note on the development plan notation:

- R denotes that a task is required or implicitly required.
- S denotes that a task falls under suggested features.
- Where applicable, tasks are also annotated with the feature they correspond to on the Suggested Game Features page on Canvas, and whether they are advanced or basic (A or B, respectively).

Milestone 1: Skeletal Game

For milestone 1 [M1], you should have a basic version of your target game. It should incorporate basic rendering, input-driven response, 2D motion, correct basic collision handling, event-driven/random response, and a minimal set of assets. Feel free to use the assignment 0 and 1 templates as a starting point for your game code.

Week 1

UI

Placeholder for UI/HUD elements - **R**

Implement basic graphics for player, enemies, and rooms- **R**

Utility

Add interaction functionality - **S27B** (buildup to M3)

^ this will initially be used for items in M2, and later, for dialogue in M3

Gameplay Mechanics

Add functionality for player movement from keyboard input for each direction - **R**

Add functionality to take user input for LMB to fire - **R**

Add functionality to take user input for mouse for aim - **R**

Add functionality to track player health - **R**

Add functionality to track enemy health - **R**

^ health will be a small number of hit points for enemies and players (e.g. player starts with 2)

AI

Implement pathfinding system for enemy to track player - **S14B**

Geometry/physics

Create a functional hitbox for the main character to collide with solid objects (walls, enemies, etc.) - **R**

Create reusable wall objects with collision - **R**

Implement collision and hitbox for enemy - **R**

Add a hitbox to bullet and functionality to destroy bullet on collision with solid object/enemy - **R**

Art/Sprites

Apply interpolation to character's movement - **R**

Apply interpolation to enemy's movement - **R**

Apply interpolation to bullet's movement - **R**

Testing

Develop test plan - **R**

Week 2

Gameplay Mechanics

Implement damage application on player by enemy - **R**

Implement damage application on enemy from bullet - **R**

Develop algorithm for procedural generation of a floor - **R**

Add basic death handling for player (just full reset) - **R**

Add simple death handling for when enemy HP is reduced to 0 (just disappear) - **R**

Add pausing functionality - **R**

Geometry/physics

Create reusable floor objects - **R**

Well-defined game-space boundaries (at least one room) - **R**

Add particles appearing from the main character when damage occurs which obey gravity - **S8B**

Art/Sprites

Add texture to all geometry for walls, character, objects, and enemy - **R**

Testing

Testing/optimizing, keep track of bugs and fixes - **R**

Other

Create demo Video - **R**

Milestone 2: Minimal Playability

For this milestone, you should continue to support all required skeletal game features and fix any game-stopping bugs (P0 and P1). You should augment those features with core gameplay logic, incorporate additional assets and play elements that allow for non-repetitive gameplay, fix all known bugs, and perform playability testing. There should be 2 minutes of non-repetitive gameplay (i.e. multiple rooms)

Week 1

UI

Implement main menu functionality (on start)

^ note that this doesn't include implementing the button actions, just make them clickable and make them output something to use later

- Start button - **R**
- Test room button
- Help button - **R**
- Save & Quit button
- Shop button

Implement pause menu

- Resume button
- Quit button
- Help button?

Implement help screen - **R**

^ Plan B: have a basic graphic in the first room showing keybindings and mechanics, i.e. how to move, shoot, use items, scrap

Add game over screen - **R**

Add permanent upgrade shop screen

^ can be worked on in parallel to shop button in main menu and linked after

Add inventory tab on HUD

Utility

Implement procedural generation for a floor - **R**

Add reload capability (saving permanent upgrades and game state) - **S19B**

Add save & quit functionality - **S19B**

Add a test room (big room) with functionality within code to add enemies

Geometry/Physics

Implement mesh-based collision - **R**

Gameplay Mechanics

Add functionality for item drops from enemies (probability-based)

Implement ability to pick up items (max 2, not changeable for now)

Implement one item of each rarity

Add one enemy (ranged)

Add option to scrap item and increases player's scrap count

AI

Pathfinding should already be done, but if it's not, do it - **R**

Implement basic decision tree for two AI enemies (melee and ranged) - **R**

Art/Sprites

Implement basic assets for background - **R**

Implement basic assets for walls - **R**

Implement basic assets for player - **R**

Implement basic assets for enemies - **R**

Create title asset

Bugs/Testing

Fix bugs

Develop test plan

Week 2

UI

Create overlay for FPS counter - **R**

Utility

Implement FPS counter - **R**

Geometry/Physics

Bug fixes where necessary

Gameplay Mechanics

Implement full health system for enemies and player - **R**

Implement item effects on player - **R**

Implement dying (shows game-over screen, returns to title screen) - **R**

Implement full health system for enemies and player - **R**

Add functionality for a basic shop from the menu screen with three upgrades

- add item slot
- upgrade base damage
- upgrade base defense
- upgrade health

Add functionality for adding an item slot

Implement shop functionality with permanent upgrades

AI

Bug fixes where necessary

Art/Sprites

Add damage sound effect - **S23B**

Add shooting sound effect - **S23B**

Add melee sound effect - **S23B**

Add basic background music - **S23B**

^ 2 tracks, one for combat and one for post-combat

Implement basic attack animations for the two enemies and player - **R**

- player firing: particle effects or explosion
- enemy melee: swing animation
- enemy firing: particle effects or explosion

Create/source asset for new enemy

Bugs/Testing

Execute tests and fix bugs

Milestone 3: Playability

For this milestone, you should have a completely playable game, continue to support all required features from prior milestones, and have fixed the majority of your latent bugs. In addition, you should now also include advanced features such as detailed geometry, non-linear motion, and time-stepping based physics. Your game should provide robust continuous play with no memory leaks, crashes or glitches.

Key to this milestone is that you test the playability of all new features and ensure alignment with your team's game development plan. You are encouraged to ask people outside your team to playtest your game, so you can fine tune your gameplay and interface.

Week 1

UI

Simple rendering special effect (when the player gets hit, when enemies die, while collecting items) - **S1B**

Implement particle effect on player/weapon/bullet when items are attached - **S5A**

Implement semi-transparent text box at the bottom of the screen - **S27B**

Utility

Add 2 NPC rooms to procedural generation (probability of spawning once per run) - **R**

Add 1 previous player death room to procedural generation - **S27B**

Add boss room to procedural generation - **R**

Add 2 more stages to procedural generation algorithm

^ currently, they will all be the same apart from their final boss

Serialize player deaths for previous player death room - **S27B**

Implement text rendering library for dialogue - **S27B**

Implement camera tracking player in boss rooms (boss rooms are larger) - **S21B**

Gameplay Mechanics

Implement peaceful room (NPC, previous death) framework - **R**

Implement 15 items - **R**

Implement 4 more unique enemy types with their own attack patterns

^ New enemies will complement old enemies on newer stages, and they will have more health and more complex attack patterns

AI

Implement advanced path finding for enemies - **S15A**

Art/Sprites

Design trader room layout - **R**

Design previous player death room

Design/source assets for 15 items

Design/source assets for 4 enemies

Design 1 NPC room - **S27B**

NPCs:

- Design/source assets for 1 NPCs - **S27B**
- Design the story for 1 NPCs - **S27B**
- Design the rooms for 1 NPCs - **S27B**

Bosses:

- Design/source assets for 2 bosses - **R**
- Design the story for 2 bosses - **R**
- Design the rooms for 2 bosses

Bugs/Testing

Execute tests and fix bugs

Week 2

UI

Add trader shop screen

Utility

Implement advancing between stages - **R**

Implement a basic spawn rate for enemies per room throughout the 3 stages

Gameplay Mechanics

Implement boss attacking pattern - **R**

Implement function to talk to a npc - **R**

Implement generation of items for sale in trader room - **R**

Implement buying item from trader with current run's scrap - **R**

Implement dropping items on the ground/scrapping items to make space - **R**

Design the attacking style, attack pattern for final boss - **R**

Art/Sprites

Design/source the asset for final boss - **R**

Design the story for final boss - **R**

Design the room for final boss

Design/source assets for 2 NPCs - **S27B**

Design the story for 2 NPCs - **S27B**

Design the rooms for 2 NPCs - **S27B**

Write dialogue exchange tree for 3 NPCs (dialogue from NPCs, and dialogue option from player) - **S27B**

Write dialogue exchange tree for 3 bosses (dialogue from bosses, and dialogue option from player) - **S27B**

select and add background music for regular room/boss/npc - **S26A** (buildup for M4)

write script for winning cutscene - **R**

Add sfx for dialogue (like undertale dialogue sound effects)

Bugs/Testing

Execute tests and fix bugs

Milestone 4: Final Game

The final version of your game should support robust and continuous gameplay and integrate advanced game elements. It should be self-contained, i.e., players should be able to play the game with no outside help or instructions other than those provided by the game itself. You should implement one or more additional advanced gameplay features (see below) and you are welcome to incorporate advanced features using third-party libraries (verify with TA). Your final

video will demonstrate your final game, highlight the advanced features, and provide a review of your game's development and evolution.

Key to this final milestone is that you test the playability of all new features and ensure alignment with your team's game development plan. You are encouraged to ask people outside your team to playtest your game, so you can balance and fine tune your gameplay and interface.

Week 1

UI

Ensure resolution and aspect ratio are consistent across different machines/displays - **R**

Ensure all inputs are appropriately handled, including player control, item use, pause, switching windows, minimizing the window, hitting invalid inputs. - **R**

Finalize the tutorial page from the main screen to cover all aspects of the game. - **R**

Add tutorial page button in pause menu - **R**

Utility

Work on ensuring no crashes, glitches, lack of smoothness. The game should run consistently and as expected. - **R**

Fix any memory hoarding - **R**

Fix any memory leaks - **R**

If necessary, create additional layout options for procedural generation to ensure gameplay variety. - **R**

If necessary, create additional enemy types to ensure that enemy generation is sufficiently varied to produce fresh gameplay. - **R**

Geometry/Physics

Implement more advanced item-based attack components (ex. bouncing projectiles, scrap grenades, sprayfire, enemy-seeking projectiles, etc.) - **S11A**

Gameplay Mechanics

Finalize interactive components based on the implementation of most other mechanics (eg. player speed is not too slow to get away, not so fast that enemy pursuit is trivial). - **R**

Balance player and enemy health

Balance items

Balance enemy per room ratio

AI

Implement enemy group coordination for more realistically challenging gameplay (by making enemy training more difficult, for example). - **S14A**

Art/Sprites

All sprites should be fully implemented without bugging to a reasonable extent of animation. - **R**

Add game state-based sprite varieties for the main character (i.e. appearance changing based on current possessed items). - **S1B**

If necessary, design/source assets for additional enemy types - **R**

Add dialogue tree options and add more dialogue - **S27B**

Bugs/Testing

Extensive playtesting by ourselves and others to hunt for unexpected bugs and ensure all components are fully operational and that the game can be played without prior knowledge. - **R**

Week 2

UI

After all implementations, ensure that the game runs in real-time (no lag or jumpiness, consistent frame rate). - **R**

Conduct performance testing to prepare for our discussion of it in the demo video. - **R**

Utility

Fix any remaining crashes as discovered from ongoing playtesting. - **R**

Geometry/Physics

Finalize new and existing physics systems, ensure no bugs remain. - **R**

Gameplay Mechanics

Ensure polished tutorial concisely presents game instruction to a player with no prior knowledge based on previous week's playtesting. - **R**

AI

Bug fixes. - **R**

Art/Sprites

Debug extra state-based sprites. - **S1B**

Last chance to fix any other lasting bugs. - **R**

Proofread dialogue - **S27B**

Bugs/Testing

Fix any remaining bugs in the project, by order of priority if they can't all be done. - **R**

Track these bugs for reporting. - **R**